

# DevAgent 智能体使用说明与应用案例

安装配置 · CLI 命令详解 · IDE 集成 · 应用案例演示

开发组织：DevAgent 开发团队

# DevAgent 智能体使用说明与应用案例

安装配置 · CLI 命令详解 · IDE 集成 · 应用案例演示

开发组织：DevAgent 开发团队

日期：2026-06-04

## 1. 安装与配置

### 1.1 环境要求

### 1.2 安装步骤

```
# Step 1: 克隆项目（或解压源码包）
cd DevAgent

# Step 2: 创建虚拟环境
python -m venv venv
source venv/bin/activate # macOS/Linux
# 或: venv\Scripts\activate # Windows

# Step 3: 安装依赖
pip install -r requirements.txt
pip install -e ".[all]"

# Step 4: 配置 API 密钥
cp .env.example .env
# 编辑 .env 文件，填入 API Key:
# DEEPSEEK_API_KEY=sk-xxx
# 或
# OPENAI_API_KEY=sk-xxx
# OPENAI_BASE_URL=https://api.openai.com/v1

# Step 5: 验证安装
agent --version
# 输出: DevAgent v3.4
```

### 1.3 配置文件说明

配置文件位于 devagent/configs/config.yaml，支持以下配置项：

```
model:
  provider: deepseek      # deepseek / openai
  deepseek:
    api_key: ${DEEPSEEK_API_KEY}
    model: deepseek-chat
    base_url: https://api.deepseek.com/v1
  openai:
    api_key: ${OPENAI_API_KEY}
    model: gpt-4o
    base_url: https://api.openai.com/v1

workflow:
  max_retry: 3           # 阶段最大重试次数
  enable_quality_gates: true # 启用 LLM 质量门检查
  use_sandbox: false     # 启用 Docker 沙箱（需要 Docker）

tools:
  pytest_timeout: 120    # 测试超时（秒）
  shell_timeout: 60      # Shell 命令超时（秒）

sandbox:
  provider: auto         # docker / podman / auto
  docker:
    image: python:3.12-slim
    memory_limit: 2g
    cpu_limit: 2.0
```

## 2. CLI 命令详解

### 2.1 基本语法

```
agent --mode <MODE> [OPTIONS]
```

模式 (--mode / -m)：

- full 全流程开发：需求分析→设计→编码→测试→修复
- design 仅分析+设计：需求分析→架构设计
- implement 代码生成：需求→设计→编码→测试
- test 独立测试：对已有代码生成测试并执行
- debug 独立调试：对失败测试进行根因分析
- repair 独立修复：对已知 Bug 进行修复+回归
- agentic 自主模式：Agent 自行决定执行策略

### 2.2 常用参数

### 2.3 交互模式说明

## 3. IDE 集成方法

### 3.1 VSCode 扩展

```
# 安装 VSCode 扩展
cd vscode_extension
npm install
# 按 F5 启动扩展（开发模式）
# 或打包为 .vsix 后安装：code --install-extension devagent-*.vsix

# 使用方式：
# 1. 右键菜单 → DevAgent 子菜单
# - 全流程开发
# - 分析设计
```

```
# - 代码生成
# - Bug 修复
# - 审核审批
# - 暂停/恢复
#
# 2. Command Palette (Cmd+Shift+P) → DevAgent 命令
#
# 3. 交互审核：
#   Agent 完成每个阶段后，右下角自动弹窗
#   [批准] / [要求修改] / [拒绝]
```

### 3.2 IntelliJ IDEA / JetBrains 插件

```
# 构建插件
cd intellij_plugin
./gradlew buildPlugin
# 产物在 build/distributions/devagent-*.zip
# 或通过 JetBrains Marketplace 安装

# 使用方式：
# Tools → DevAgent → 选择模式
```

### 3.3 Eclipse 插件

```
# 构建插件
cd eclipse_plugin
# 使用 Eclipse PDE 导出为可安装插件
# plugin.xml 定义了 DevAgent 工具栏按钮和菜单项
```

### 3.4 REST API 集成（通用）

```
# 启动 API 服务器
devagent-api
# 或指定交互模式
devagent-api --interactive full

# 服务端点：
# http://127.0.0.1:8911/docs    Swagger API 文档
# http://127.0.0.1:8911/dashboard Web 仪表盘
# ws://127.0.0.1:8911/api/v2/tasks/{id}/interactive 实时 WebSocket

# 提交任务 (REST)
curl -X POST http://127.0.0.1:8911/api/v2/tasks/agentica \
  -H "Content-Type: application/json" \
  -d '{"task": "agentica", "input": "...", "code": "...", "output": ".../outputs/task1"}'

# 查询任务状态
curl http://127.0.0.1:8911/api/v1/tasks/{task_id}

# 审核响应
curl -X POST http://127.0.0.1:8911/api/v2/tasks/{task_id}/review/respond?review_id=xxx&decision=approve&feedba...
```

## 4. 应用案例一：Python 计算器全流程开发

本案例演示从需求文档到可运行项目的完整自动化开发流程。

### 4.1 输入：需求文档 (requirements.md)

```
# 简易计算器应用需求文档

## 概述
构建一个基于 Python 的计算器应用，支持基本算术运算。

## 功能需求
1. 基本运算：支持加法 (+)、减法 (-)、乘法 (*)、除法 (/)
```

2. 输入处理：接受字符串格式的数学表达式（例如 "3 + 4 \* 2"）
3. 运算符优先级：遵循标准数学优先级（乘除优先于加减），支持括号
4. 错误处理：对除零和无效输入返回清晰的错误信息
5. 结果展示：计算结果保留 2 位小数

#### ## 非功能需求

- 使用 Python 3.10+ 编写
- 包含完整的 docstring 文档注释
- 包含完整的类型标注 (type hints)

#### ## 验收标准

- 3 + 4 = 7.00
- 10 - 5 = 5.00
- 6 \* 7 = 42.00
- 15 / 3 = 5.00
- 10 / 0 返回清晰错误信息
- "abc + 5" 返回清晰错误信息
- 3 + 4 \* 2 = 11.00 (优先级正确)
- (3 + 4) \* 2 = 14.00 (括号支持)

## 4.2 执行命令

```
agent --mode full --input requirements.md --output outputs/calculator
```

## 4.3 执行过程与输出

Agent 自动执行以下阶段：

阶段 1：需求分析 → 生成 structured\_requirements.json

输出产物: outputs/calculator/01\_requirements/

- ├── requirement\_specification.md # SRS 软件需求规格说明 (IEEE 830)
- └── structured\_requirements.json # 结构化需求数据
- 项目摘要 (Calculator App)
- 领域模型: Expression, Token, Calculator
- 5 个 FR (功能需求)，含验收标准
- 3 个 NFR (非功能需求)，含可测量指标
- 6 个用例 (Use Case)，含主流程和备选流程
- 风险评估: 除零/无效输入为高风险
- 安全需求: 输入验证 (OWASP A1)

阶段 2：架构设计 → 生成设计文档和图表

输出产物: outputs/calculator/02\_design/

- ├── design\_specification.md # SDD 软件设计说明 (IEEE 1016)
- └── diagrams/
  - ├── class\_diagram.png # 类图: Calculator, Tokenizer, Parser, Evaluator
  - ├── er\_diagram.png # N/A (calculator 无持久化)
  - ├── sequence\_diagram.png # 序列图: 表达式解析流程
  - ├── activity\_diagram.png # 活动图: 计算流程 (含除零判断)
  - ├── usecase\_diagram.png # 用例图: User → Calculate
  - ├── context\_diagram.png # 系统上下文图
  - ├── deployment\_diagram.png # 部署图
  - └── state\_diagram.png # 状态图: Tokenizer 状态转换

设计要点:

- 架构模式: Parser-Evaluator 模式
- 模块: tokenizer, parser, evaluator, calculator
- 类: Token, ASTNode, Parser, Evaluator, Calculator
- API: Calculator.evaluate(expression: str) -> float

阶段 3：代码生成 → 生成完整项目

输出产物: outputs/calculator/03\_implementation/

- ├── pyproject.toml # 项目元数据 + 构建配置
- ├── requirements.txt # 生产依赖
- ├── requirements-dev.txt # 开发依赖
- └── Dockerfile # 多阶段构建 + 非 root 用户

```

├── .env.example          # 环境变量模板
├── .gitignore
├── Makefile              # install, test, lint, run, clean
├── README.md             # 项目说明
├── src/
│   ├── __init__.py
│   ├── config.py         # pydantic-settings 配置
│   ├── exceptions.py     # 自定义异常层次结构
│   ├── logging_config.py # JSON 结构化日志
│   ├── models/
│   │   ├── __init__.py
│   │   ├── token.py      # Token, TokenType 枚举
│   │   └── ast_node.py   # ASTNode, NumberNode, BinOpNode
│   ├── services/
│   │   ├── __init__.py
│   │   ├── tokenizer.py  # 词法分析器
│   │   ├── parser.py     # 语法分析器 (递归下降)
│   │   ├── evaluator.py  # 表达式求值器
│   │   └── calculator.py # 计算器门面类
│   └── main.py           # CLI 入口 + /health 端点

```

代码质量指标:

- 类型标注覆盖率: 100% (所有公共函数)
- docstring 覆盖率: 100% (Google 风格)
- 自定义异常: CalculationError, DivisionByZeroError, ParseError
- 安全: 0 eval() / 0 shell=True / 全部参数化 / 密钥从环境变量读取

#### 阶段 4：测试生成与执行 → 测试套件 + 报告

输出产物: outputs/calculator/04\_tests/

```

├── tests/
│   ├── conftest.py      # 共享 fixtures
│   ├── test_tokenizer.py # 词法分析测试 (12 用例)
│   ├── test_parser.py   # 语法分析测试 (10 用例)
│   ├── test_evaluator.py # 求值器测试 (8 用例)
│   ├── test_calculator.py # 集成测试 (验收标准映射)
│   └── test_security.py  # 安全测试
├── pytest_result.json    # 测试结果 (collected/passed/failed)
├── test_strategy.json    # 测试策略 (等价类 + 边界值)
└── test_execution_report.md # IEEE 829 测试报告

```

测试结果摘要:

```

Tests Collected: 35
Passed: 35
Failed: 0
Line Coverage: 96.5%
Branch Coverage: 89.2%

```

#### 4.4 最终交付产物总览

```

outputs/calculator/
├── execution.log          # 完整执行日志
├── 01_requirements/
│   ├── requirement_specification.md # IEEE 830 SRS
│   └── structured_requirements.json
├── 02_design/
│   ├── design_specification.md     # IEEE 1016 SDD
│   └── diagrams/                   # 8-12 张 PNG 图表
├── 03_implementation/
│   ├── src/                       # 生产级源代码 (15+ 文件)
│   ├── tests/                     # 测试套件 (10+ 文件)
│   ├── Dockerfile
│   ├── Makefile
│   └── README.md
├── 04_tests/
└── tests/                        # 完整测试套件

```

```

|   |—— pytest_result.json
|   |—— test_execution_report.md   # IEEE 829 测试报告
|—— 06_reports/
|   |—— executive_report.md       # 执行摘要 + 质量仪表盘

```

## 5. 应用案例二：Bug 修复 workflow

本案例演示对已有代码的 Bug 定位和自动修复流程。

### 5.1 场景设定

已知一个 Python 项目存在以下 Bug：

```

# buggy_code/src/math_ops.py

def divide(a, b):
    """BUG: 未检查除数是否为零的情况"""
    return a / b # 当 b=0 时抛出 ZeroDivisionError

# 测试文件 tests/test_math.py
def test_divide_by_zero():
    with pytest.raises(ValueError): # 期望 ValueError
        divide(10, 0) # 实际抛出 ZeroDivisionError

```

### 5.2 执行命令

```
agent --mode repair --workspace ./buggy_code/ --output outputs/repair
```

### 5.3 Agent 执行过程

第 1 步：测试执行 → 发现失败

```

[TestRunner] Collected 5 tests
[TestRunner] PASSED: test_add, test_multiply, test_divide_normal, test_safe_divide
[TestRunner] FAILED: test_divide_by_zero
AssertionError: DID NOT RAISE ValueError
Actual: ZeroDivisionError("division by zero")

```

第 2 步：三层故障定位

```

[SBFL] Ochiai 分析完成 — Top 5 可疑语句：
Rank 1: src/math_ops.py:9 in divide() — suspiciousness=0.95
Rank 2: src/math_ops.py:8 in divide() — suspiciousness=0.85

[StaticAnalyzer] 发现风险点：
[boundary] src/math_ops.py:9 — Division by variable 'b' without zero check
[exception] src/math_ops.py:9 — / operator without try/except

[LLM Fusion] 置信度: high
Bug 定位: src/math_ops.py:9 in divide()
根因: divide() 未检查除数 b 是否为零
修复方案: 在 division 前添加 b == 0 检查, raise ValueError

```

第 3 步：自动修复

```

[RepairAgent] 生成修复补丁:

--- a/src/math_ops.py
+++ b/src/math_ops.py
@@ -7,3 +7,5 @@
def divide(a, b):
- return a / b
+ if b == 0:
+     raise ValueError("Division by zero is not allowed")
+ return a / b

修复类型: boundary_fix

```

风险等级: low  
修改文件: 1

## 第 4 步：回归验证

[TestRunner] 回归测试执行完成：

Collected: 5  
Passed: 5  
Failed: 0  
Coverage: 94.3%

[RepairAgent] 修复成功！回归测试全部通过。

## 5.4 修复产物

```
outputs/repair/05_repair/
├── repaired_code/
│   └── math_ops.py          # 修复后的代码
├── patch.diff              # Unified diff 补丁
├── repair_result.json      # 修复详情 (fix_type, changes, risk)
├── repair_report.md        # 修复报告
└── debug_analysis_report.md # 调试分析报告
```

# 6. Docker 部署

```
# 使用 Docker Compose
cd DevAgent
docker compose up -d

# 服务将在以下端口可用：
# http://localhost:8911/docs    Swagger API 文档
# http://localhost:8911/dashboard Web 仪表盘
# ws://localhost:8911/...      WebSocket 实时通信

# 查看日志
docker compose logs -f

# 停止服务
docker compose down
```

# 7. 输出目录结构

```
outputs/{task_id}/
├── execution.log          # 完整执行日志
├── 01_requirements/
│   ├── requirement_specification.md # IEEE 830 SRS 文档
│   └── structured_requirements.json # 结构化需求数据
├── 02_design/
│   ├── design_specification.md      # IEEE 1016 SDD 文档
│   ├── class_diagram.mmd           # 类图 (Mermaid)
│   ├── er_diagram.mmd             # ER 图 (Mermaid)
│   ├── activity_diagram_1.puml     # 活动图 (PlantUML)
│   ├── usecase_diagram.puml       # 用例图 (PlantUML)
│   └── diagrams/                  # 渲染后的 PNG 图像
├── 03_implementation/
│   ├── src/                      # 生成的源代码
│   ├── tests/                   # 生成的测试文件
│   ├── Dockerfile               # 容器构建文件
│   ├── .env.example             # 环境变量模板
│   ├── Makefile                 # 构建脚本
│   └── README.md                # 项目说明
└── 04_tests/
```

```

|   |—— tests/                # 测试文件
|   |—— pytest_result.json    # 测试执行结果
|   |—— test_strategy.json    # 测试策略
|   |—— test_execution_report.md # IEEE 829 测试报告
|—— 05_repair/ (如有修复)
|   |—— repaired_code/        # 修复后的代码
|   |—— patch.diff            # 修复补丁
|   |—— repair_result.json    # 修复详情
|   |—— repair_report.md      # 修复报告
|   |—— debug_analysis_report.md # 调试分析报告
|—— 06_reports/
|   |—— executive_report.md    # 执行摘要 + 质量仪表盘

```

## 8. 常见问题 (FAQ)

**Q: 如何切换大语言模型？**

A: 修改 .env 文件中的 API Key 配置，或使用 --provider 命令行参数。配置文件位于 devagent/configs/config.yaml。

**Q: 生成的代码需要手动修改吗？**

A: DevAgent 生成的代码是可直接运行的（含类型标注、错误处理、日志、测试），但建议进行代码审查（Code Review）以确保符合团队规范。

**Q: 如何处理 LLM 输出不稳定（同一输入产生不同代码）？**

A: 可以在配置文件中将 temperature 设置为 0（或接近 0 的值 0.1），降低输出的随机性。

**Q: 沙箱模式是否必需？**

A: 非必需。沙箱模式提供了额外的安全隔离层，可在配置中设置 sandbox.provider=auto 自动检测可用环境，或在无 Docker 时自动回退到 Local 模式。

**Q: 如何扩展对新编程语言的支持？**

A: 在 agents/code\_agent.py 中为新语言编写 CODE\_PROMPT，添加项目模板和测试框架支持。详见系统设计方案文档第 5.1 节。

**Q: 生成失败如何排查？**

A: 检查 outputs/{task\_id}/execution.log 查看完整执行日志，包含每个阶段的详细追踪信息。

| 依赖项    | 最低版本       | 说明     |
|--------|------------|--------|
| Python | 3.11+      | 运行环境   |
| pip    | 23.0+      | 包管理器   |
| Docker | 24.0+ (可选) | 沙箱隔离执行 |
| Git    | 2.40+      | 版本控制集成 |

| 参数               | 简写 | 默认值       | 说明                               |
|------------------|----|-----------|----------------------------------|
| --mode           | -m | agentic   | 任务模式                             |
| --input          | -i |           | 输入文件路径（design/implement/full 模式） |
| --workspace      | -w | .         | 工作空间目录                           |
| --output         | -o | ./outputs | 产物输出目录                           |
| --max-iterations |    | 50        | 最大迭代次数                           |
| --interactive    | -l | off       | 交互模式：full/approval/observe/off   |
| --provider       |    | (配置文件)    | Provider 覆盖：openai/deepseek      |



|           |    |       |           |
|-----------|----|-------|-----------|
| --verbose | -v | false | 详细输出模式    |
| --config  |    |       | 自定义配置文件路径 |

| 模式       | 行为                                          |
|----------|---------------------------------------------|
| off      | 完全自主 — Agent 不等待人工输入，所有审核自动通过               |
| observe  | 观察模式 — Agent 实时推送进度，但不需要人工审批                |
| approval | 审批模式 — Agent 执行危险操作前弹窗等待确认（文件删除、git push 等） |
| full     | 完整交互 — 每个阶段完成后等待人工审核（批准/要求修改/拒绝）+ 危险操作审批    |